

LESSON 09: FIXED-POINT ARITHMETIC

One day, you may have the great fortune of implementing a digital filter on an embedded device where you only have a few microseconds to filter before the next sample. In such a scenario, performing floating-point operations is impractical due to how much slower they are to ***Multiply–Accumulate (MAC)*** operations. To benefit from MAC operations, we turn to fixed-point arithmetic, but in the process, we open Pandora’s box and must account for the substantial quantization error we’ve introduced. Luckily for us, MATLAB offers us a Fixed-Point Designer¹ for optimizing and implementing fixed-point algorithms.

Constructing Fixed-Point Types

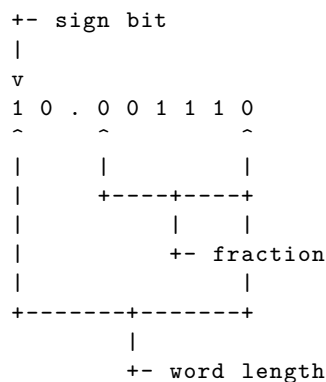


Figure 1: Fixed-Point Value

First, we must tell MATLAB the type of fixed-point value we wish to construct:

```

1 T = numerictype(true, 32, 30)
2
3
4     DataTypeMode: Fixed-point: binary point scaling
5     Signedness: Signed
6     WordLength: 32
7     FractionLength: 30

```

¹<https://www.mathworks.com/help/fixedpoint/index.html>

In our case, we've constructed a signed 32-bit fixed-point value with a fraction length of 30 bits, giving us a range from -2 to $2 - 2^{-30}$ in increments of 2^{-30} .

Now, to convert a `double` to a fixed-point value, we call `fi()`:

```
1 mu_0 = fi(4 * pi * 1e-7, T)
_____
1 mu_0 =
2
3     1.2564e-06
4
5         DataTypeMode: Fixed-point: binary point scaling
6         Signedness: Signed
7         WordLength: 32
8         FractionLength: 30
```

Set Fixed-Point Math Settings

At times, we may wish to change MATLAB's default behavior when performing fixed-point math. Say we wanted to have two's complement overflow as opposed to saturation:

```
1 saturate = 1 / mu_0
_____
1 saturate =
2
3     2.0000
4
5         DataTypeMode: Fixed-point: binary point scaling
6         Signedness: Signed
7         WordLength: 32
8         FractionLength: 30
```

```

1 mu_0.OverflowMode = 'Wrap';
2 wrap = 1 / mu_0

```

```

1 wrap =
2
3     1.9096
4
5     DataTypeMode: Fixed-point: binary point scaling
6     Signedness: Signed
7     WordLength: 32
8     FractionLength: 30
9
10    RoundingMethod: Nearest
11    OverflowAction: Wrap
12    ProductMode: FullPrecision
13    SumMode: FullPrecision

```

We can also apply these properties to new fixed-point types with `fimath()`:

```

1 F = fimath('OverflowMode', 'Wrap');
2 mu_r = fi(0.34, T, F)

```

```

1 mu_r =
2
3     0.3400
4
5     DataTypeMode: Fixed-point: binary point scaling
6     Signedness: Signed
7     WordLength: 32
8     FractionLength: 30
9
10    RoundingMethod: Nearest
11    OverflowAction: Wrap
12    ProductMode: FullPrecision
13    SumMode: FullPrecision

```

Fixed-Point Filter Design

For this part of the lesson, we'll be working with the brake pressure sensor data from Assignment 07:²

²https://raw.githubusercontent.com/jacobkoziej/jk-ece210/master/src/assignments/07-under-pressure.d/40p_1000ms.csv

```
1 RESOLUTION = 2^12;
2
3 fs = 80e3;
4
5 M = readmatrix("40p_1000ms.csv");
6
7 t = M(:, 1) ./ fs;
8 s = M(:, 2) ./ (RESOLUTION - 1);
```

Since fixed-point values are objects, we can't use our standard filtering functions on them, so we turn to filter objects:

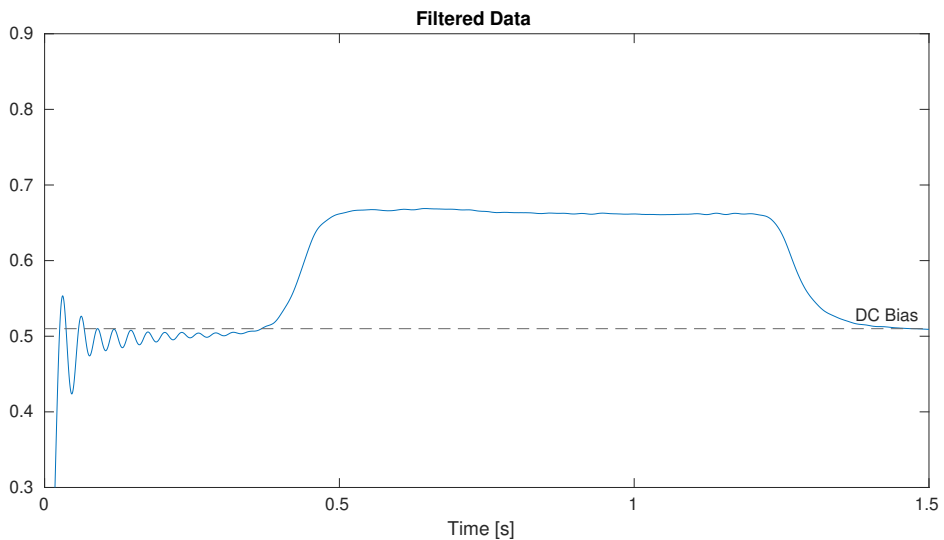
```
1 Fp = 35;
2 Fs = 45;
3 Rp = 1;
4 Rs = 40;
5
6 D = fdesign.lowpass(Fp, Fs, Rp, Rs, fs);
7 LPF = design(D, 'ellip', 'SystemObject', true);
8
9 LPF.Structure = 'Direct form I';
10 LPF.DenominatorAccumulatorDataType = T;
11 LPF.OutputDataType = T;
```

We can then apply the filter to our quantized data like so:

```
1 s_filtered = LPF(fi(s, T));
2
3 figure;
4 plot(t, s_filtered);
5 title('"Filtered" Data');
6 xlabel('Time [s]');
```

Aside—

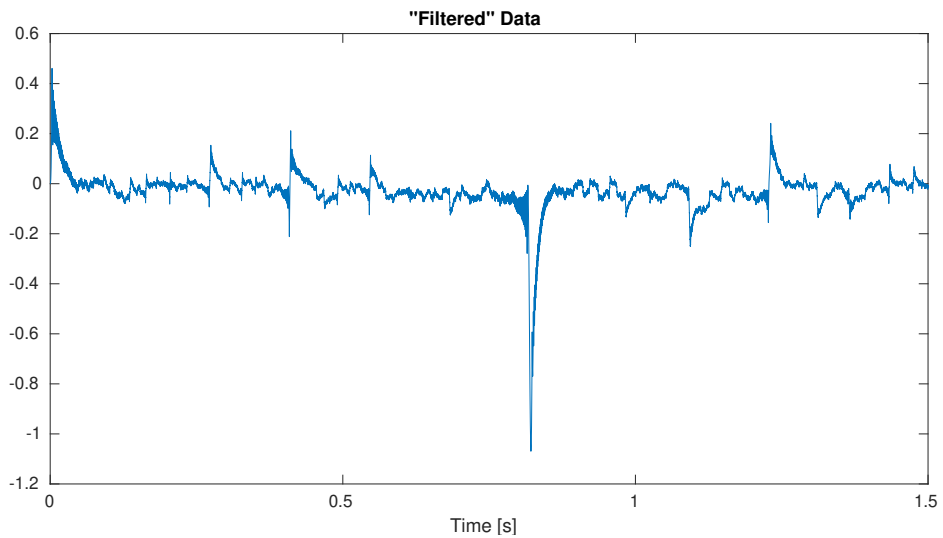
We use the Direct Form I structure over Direct Form II to gain more numerical stability at the cost of using double the amount of state variables.



Let's half our bit allocation to see how our filter performs:

```

1  T = numerictype(true, 16, 14)
2
3  D = fdesign.lowpass(Fp, Fs, Rp, Rs, fs);
4  LPF = design(D, 'ellip', 'SystemObject', true);
5
6  LPF.Structure = 'Direct form I';
7  LPF.DenominatorAccumulatorDataType = T;
8  LPF.OutputDataType = T;
9
10 s_filtered = LPF(fi(s, T));
11
12 f = figure;
13 plot(t, s_filtered);
14 title('Filtered Data');
15 xlabel('Time [s]');
```



Ouch, it looks like the quantization error was too much to handle! To combat this, we can decrease our filter's requirements to avoid higher orders and hence numerical instability in the form of overflows, but keep in mind this isn't a full-proof solution.